

Adversarial Attacks and Defenses on Graph-aware Large Language Models (LLMs)

Anonymous submission

Abstract

Large Language Models (LLMs) are increasingly coupled with graph-structured inputs to perform node classification, a task historically addressed by Graph Neural Networks (GNNs). Although this coupling exploits relational signal to improve predictive accuracy, the adversarial robustness of the resulting models has not been characterized. We provide the first systematic robustness analysis of graph-aware LLMs by transferring adversarial attack procedures developed for graph-based models—covering both poisoning (perturbations injected at training time) and evasion (perturbations injected at inference time)—to two representative architectures, LLAGA (Chen et al. 2024) and GRAPH PROMPTER (Liu et al. 2024). We further identify a previously undocumented attack surface in LLAGA, in which an adversary injects malicious nodes into placeholder positions of the node sequence template and thereby induces substantial accuracy degradation. Our controlled evaluation shows that specific graph-encoding design choices measurably increase attack effectiveness, and yields three concrete findings: (1) the node sequence template in LLAGA amplifies its susceptibility to structural perturbation; (2) the GNN encoder in GRAPH PROMPTER confers greater robustness; and (3) both architectures remain susceptible to imperceptible perturbations of the textual node features. Finally, we introduce GALGUARD, an end-to-end defense framework that composes an LLM-based feature-correction module—to counter feature-level perturbations—with adapted GNN structural defenses that counter edge-level perturbations¹

1 Introduction

Graphs, defined by a set of nodes and a set of edges that respectively encode entities and their pairwise relationships, provide a unifying formalism for structured data across numerous application domains, including social networks (Fan et al. 2019), biology (Dong, Mücke, and Khosla 2022), finance (Wang et al. 2021), and healthcare (Ahmedt-Aristizabal et al. 2021). Graph Neural Networks (GNNs) (Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2022; Veličković et al. 2018), whose message-passing operators are constructed explicitly for graph-structured inputs, attain strong performance on canonical objectives such as node classification and link prediction. More recently, large language models (LLMs)—including

GPT-4 (Achiam et al. 2023), Gemini (Team et al. 2023), LLaMA (Touvron et al. 2023; Dubey et al. 2024), Vicuna (Chiang et al. 2023), and PaLM-2 (Anil et al. 2023)—pre-trained on large-scale text corpora, have motivated investigation into whether their learned representations can be repurposed for graph-structured objectives. The resulting class of models, which we refer to as *graph-aware LLMs* (Xie et al. 2023), constitutes a distinct research direction in which LLMs are applied to objectives conventionally solved by GNNs (Ye et al. 2023; Fatemi, Halcrow, and Perozzi 2023; Kuang et al. 2023; Zhang et al. 2023; Chen et al. 2023b, 2024). Empirically, graph-aware LLMs have been reported to exceed GNN accuracy on a range of graph objectives (Wenkel, Wolf, and Knyazev 2023; Chen et al. 2023b; Zhu et al. 2024; Chen et al. 2024), establishing them as competitive alternatives.

Despite their rapid adoption, the adversarial robustness of graph-aware LLMs has not been systematically characterized. Established adversarial attacks on GNNs—spanning poisoning (perturbations introduced during training) and evasion (perturbations introduced at inference) (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Dai et al. 2023)—assume a black-box threat model in which the adversary observes neither the model architecture nor its parameters. Such attacks target *model integrity*: they degrade predictive performance by applying bounded, low-visibility modifications to the graph structure or to node features. Whereas the GNN literature has concentrated almost exclusively on structural perturbations, the reliance of graph-aware LLMs on textual node attributes elevates imperceptible feature perturbations to a first-order threat that prior work has largely neglected.

These three observations—the migration from GNNs to LLMs for graph objectives, the black-box formulation of current GNN attacks, and the potential exposure to feature-level perturbations—motivate our central research question: *do adversarial attacks designed for GNNs transfer effectively to the emerging graph-aware LLM paradigm?* A prerequisite for answering this question is that the predictive performance of graph-aware LLMs is *contingent* on how the graph is integrated into the language model. Because LLMs process textual node attributes natively, feature encoding is straightforward; encoding the graph structure, by contrast, is nontrivial and requires dedicated mechanisms to represent relational information (Chen et al. 2024; Guo et al. 2023; Liu and Wu

¹Our code will be released upon acceptance.

2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024). Accordingly, we partition existing graph-aware methods into two families defined by their graph-encoding mechanism: those that serialize the graph into *textual descriptions* expressed in natural language, and those that employ *learned projectors* to map between the graph and text representation spaces. Since conventional text-based adversarial attacks on LLMs already suffice to compromise the former, we restrict our study to learned-projector methods and adopt two computationally efficient representatives, LLAGA (Chen et al. 2024) and GRAPHPROMPTER (Liu et al. 2024), both of which operate on a frozen pre-trained LLM.

Our evaluation demonstrates that particular graph-encoding design choices can, contrary to expectation, increase attack effectiveness. In particular, we characterize a new attack surface in LLAGA whereby an adversary induces severe accuracy degradation by injecting malicious nodes into placeholder positions of the node sequence template. We additionally propose an end-to-end graph-aware LLM defense, GALGUARD, which couples an LLM-based feature corrector with adapted GNN structural defenses to provide robustness against both perturbation modalities. To our knowledge, this is the first systematic study of the adversarial robustness of graph-aware LLMs. We summarize our contributions as follows:

- We transfer two representative GNN adversarial attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019), to graph-aware LLMs and quantify the transferability of poisoning and evasion attacks.
- We construct a taxonomy of graph-aware LLMs organized by graph-encoding mechanism, formalizing how distinct methodologies inject graph structure into the LLM.
- We identify a new attack surface in LLAGA and show that injecting malicious nodes into placeholder positions of the node sequence template degrades predictive accuracy.
- We conduct a comprehensive robustness assessment by applying imperceptible perturbations to the textual node features—a threat vector rarely considered in the GNN setting.
- We propose GALGUARD, an end-to-end defense that integrates LLM-based feature correction with adapted GNN structural defenses.

2 Graph Encoding-Based Taxonomy of LLM Adaptations

We partition graph-encoding mechanisms for LLMs into two classes: *textual descriptions of graphs*, in which graph content is serialized into natural language, and *learned graph projectors*, in which a trainable projector maps graph data into embeddings or structured token forms that the LLM can consume directly. We summarize the representative methods in each class below; Figure 5 illustrates the two schemes. A complete treatment appears in Appendix A.1, and related work on GNN attacks in Appendix A.2.

Graph Representation as a Textual Description. Methods in this class serialize graph content into natural language so

that the LLM processes it as text, typically under task-specific query instructions (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a; Zhang 2023). In the multimodal setting, Liu et al. (2023) and Su et al. (2022) aligned molecular graphs with their textual descriptions. Huang et al. (2023a) further examined the injection of graph structure into LLM prompts and reported that LLMs treat graph-related prompts as free-form contextual text rather than as explicit structural objects, which underscores the sensitivity of these methods to prompt design (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023; Bi et al. 2024; Pan et al. 2024; He et al. 2024). A parallel line of work employs LLMs to augment node features or graph structure, to act as end-to-end predictors, or to serve as feature extractors for downstream GNNs (Chen et al. 2023a; Chandra et al. 2020; Chen et al. 2023b; Mavromatis et al. 2023). For the inference stage, Zhu et al. (2024) proposed a fine-tuning procedure for textual graphs that couples LLMs with GNNs, and Duan et al. (2023) introduced a complementary method that improves textual-graph representation learning. While these methods provide a direct route to applying LLMs on graph objectives, they tend to under-represent higher-order structural dependencies (particularly on large graphs) and exhibit strong sensitivity to prompt engineering.

Graph Representation as a Learned Projector. To overcome the representational limits of serialized text, learned-projector methods map graph data into embeddings or graph token vectors that the LLM can process directly. Representative instances include LLAGA, which pairs node-level templates with a linear projector (Chen et al. 2024), and methods that interleave GNN layers within a pre-trained LLM (Liu et al. 2024; Qin et al. 2023). Further variants encode knowledge graphs via graph neural prompts (Tian et al. 2024), extract node features through prompt-based extractors equipped with a GNN adapter (Huang et al. 2023b), or compress graph content into fixed-length prefixes using graph transformers (Chai et al. 2023; Yang et al. 2021; Peng et al. 2024).

This Work: We restrict attention to projector methods operating on a *frozen pre-trained LLM*, since freezing the backbone yields computational efficiency and eliminates the cost of full retraining (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b). We instantiate our study with two representative models: LLAGA (Chen et al. 2024), which combines node-level templates with a linear projector, and GRAPHPROMPTER (Liu et al. 2024), which combines a GNN encoder with a linear projector. We exclude textual-description methods, as they scale poorly and expose no graph-specific attack surface beyond that of standard text-based attacks.

3 Threat Model

We adopt the threat model established in the graph adversarial-attack literature and extend its scope to graph-aware LLMs (Zügner, Akbarnejad, and Günnemann 2018; Wu et al. 2019; Zügner and Günnemann 2019; Ma, Ding, and Mei 2020; Jin et al. 2021; Dai et al. 2023; Li et al. 2023). The model is consistent with realistic deployment constraints.

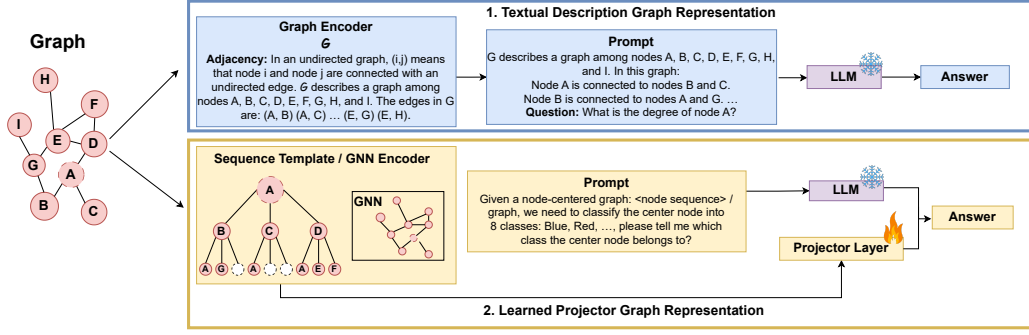


Figure 1: **Graph encoding-based adaptations of LLMs.** Under the textual-description scheme, the graph is serialized into natural language (top). Under the learned-projector scheme, the graph is encoded either by node templates that map it to token sequences or by a GNN (bottom). In both schemes the LLM parameters are frozen; only the projector that maps graph content to input tokens is optimized.

In social networks, for instance, an adversary typically controls only a small number of bot accounts, which corresponds directly to the bounded node-manipulation budget assumed here, used either to evade detection or to influence the classification of legitimate users. As a second instance, on Wikipedia, hoax articles that are sparsely connected to legitimate content can be strategically linked so as to flip the category predicted for genuine articles. The transferability of such attacks is of practical concern because graph-aware LLMs are candidates for deployment in high-stakes settings—recommendation systems, social-media analysis, and healthcare—where they may supplant GNNs. Following prior work, we specify the threat model along three axes—model access, adversarial capabilities, and attack strategies—and then state the attacker’s objective.

Model Access. The adversary operates in a black-box regime with no knowledge of the model architecture or parameters. Influence on the model is exerted solely and indirectly through modification of the training or test data.

Adversarial Capabilities. Consistent with the standard poisoning (respectively, evasion) formulation, the adversary has full access to the target model’s training (respectively, inference) data. Subject to a predefined perturbation budget and unnoticeability constraints (imposed to preclude detection), the adversary may modify existing data or inject new data.

Attack Strategies. The adversary applies two primitive operations: feature manipulation (perturbation of node features) and structure manipulation (insertion or deletion of edges).

Attacker’s Goal. The adversary’s objective is to compromise model integrity by degrading aggregate predictive performance—specifically, by increasing the misclassification rate of the node-classification model.

4 Experimental Setup

Dataset. We evaluate on three standard text-attributed graph benchmarks—Cora, Citeseer, and PubMed (Yang, Cohen, and Salakhudinov 2016)—which span small-to-medium scales and a range of edge densities and are routinely used to benchmark models of the class we study (Chen et al. 2023a, 2024; Liu et al. 2024). To assess scalability, we addition-

ally evaluate on the large-scale ArXiv benchmark (Hu et al. 2020) in a supplementary experiment (Appendix A.5). Following (Chen et al. 2024), we partition each of the three smaller datasets into train/validation/test splits with ratio 6:2:2 and ArXiv with ratio 5:2:3. Per-dataset statistics are reported in Table 1.

Table 1: Per-dataset statistics: node count, edge count, and edge density (in %).

Data	#Node	#Edge	Sparsity (%)
Cora	2708	5429	14.81
CiteSeer	3327	4614	8.34
PubMed	19717	44338	2.28

Tasks. We adopt *node classification* as the primary evaluation task; it is the canonical objective for assessing graph learning models and the standard setting in which graph adversarial attacks are evaluated (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Liu and Wu 2023).

Evaluation Metrics. We report classification *accuracy*, defined as the fraction of test nodes assigned the correct label. Each configuration is run over three independent trials, and we report the mean and standard deviation across trials. All experiments were executed on a single NVIDIA A40 GPU with 48 GB of memory.

Choice of Graph-aware LLM and Adversarial Attack Methods. We now specify the graph-aware LLM architectures and the adversarial attack procedures used throughout our robustness evaluation.

Graph-Aware LLM Methods. We instantiate two representative graph-aware LLM architectures: LLAGA (Chen et al. 2024) and GRAPH-PROMPTER (Liu et al. 2024). Both operate on a *frozen pre-trained LLM*, which keeps them computationally efficient and portable across graph objectives and LLM backbones. LLAGA encodes the graph via node-level templates followed by a linear projector, whereas GRAPH-PROMPTER encodes it with a GNN followed by a linear

Table 2: Node-classification accuracy of the graph-aware LLMs LLAGA and GRAPHPROMPTER under NETTACK and METAATTACK on Cora, Citeseer, and PubMed. Values in parentheses denote the absolute percentage-point drop relative to the clean accuracy.

Attack	Type	Cora		Citeseer		PubMed	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Clean	—	0.89 $\pm$ 0.07	0.60 $\pm$ 0.03	0.71 $\pm$ 0.04	0.70 $\pm$ 0.03	0.90 $\pm$ 0.04	0.90 $\pm$ 0.02
NETTACK	Poisoning	0.87 $\pm$ 0.04 (2%)	0.60 $\pm$ 0.03 (0%)	0.64 $\pm$ 0.05 (10%)	0.68 $\pm$ 0.02 (3%)	0.89 $\pm$ 0.03 (1%)	0.88 $\pm$ 0.02 (2%)
	Evasion	0.55 $\pm$ 0.09 (38%)	0.53 $\pm$ 0.05 (12%)	0.59 $\pm$ 0.04 (19%)	0.61 $\pm$ 0.03 (14%)	0.84 $\pm$ 0.05 (7%)	0.81 $\pm$ 0.02 (10%)
METAATTACK	Poisoning	0.79 $\pm$ 0.03 (11%)	0.58 $\pm$ 0.05 (3%)	0.63 $\pm$ 0.04 (12%)	0.65 $\pm$ 0.06 (7%)	0.89 $\pm$ 0.08 (1%)	0.85 $\pm$ 0.02 (6%)
	Evasion	0.44 $\pm$ 0.06 (51%)	0.52 $\pm$ 0.04 (13%)	0.50 $\pm$ 0.03 (35%)	0.56 $\pm$ 0.02 (22%)	0.73 $\pm$ 0.04 (19%)	0.77 $\pm$ 0.03 (14%)

projector; in both cases the projector is realized as a two-layer multi-layer perceptron (MLP). The backbone LLMs are Vicuna-7B (Chiang et al. 2023) for LLAGA and LLaMA2-7B (Touvron et al. 2023) for GRAPHPROMPTER.

Adversarial Attack Methods. Our investigation uses the poisoning and evasion variants of two representative structural attacks, NETTACK (Zügner, Akbarnejad, and Günnemann 2018) and METAATTACK (Zügner and Günnemann 2019). NETTACK is a targeted attack that computes minimal perturbations to misclassify individual nodes, whereas METAATTACK is an untargeted attack that optimizes the graph structure by treating the adjacency matrix as a hyperparameter. To obtain an untargeted evaluation from NETTACK, we follow (Li et al. 2023; Zügner and Günnemann 2019) and attack each test node in turn, aggregating the per-node outcomes. Across all experiments we cap the structural perturbation at 10% of the edges to satisfy the unnoticeability constraint of (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019). Complete procedural details are given in Appendix A.2.

5 Results

5.1 Poisoning and Evasion Attacks on Graph-aware LLMs

We evaluate poisoning and evasion attacks by transferring the perturbation procedures of NETTACK (Zügner and Günnemann 2019) and METAATTACK (Zügner and Günnemann 2019). Under the poisoning protocol for LLAGA, we derive the node-level sequences from the perturbed graph and retrain the projector on them; for GRAPHPROMPTER, we re-encode the perturbed graph with a GAT and retrain its projector. Under the evasion protocol, both models are trained on the clean graph and the perturbation is applied only at inference time.

Results. Table 2 summarizes three findings. First, poisoning attacks have limited effect on graph-aware LLMs, inducing at most an 11 percentage-point accuracy drop under either attack; we attribute this resilience to the models’ robust learned representations and to the auxiliary context supplied by the frozen LLM. Evasion attacks, by contrast, are markedly more effective, reducing accuracy by up to 51 percentage points. Second, across all datasets METAATTACK generally dominates NETTACK for both attack modalities, which we attribute to its formulation of edges as optimizable hyperparameters,

yielding small but high-impact perturbations. Third, attack effectiveness increases with graph density: for LLAGA under evasion, accuracy falls by 51 percentage points on the denser Cora graph but by only 19 percentage points on the sparser PubMed graph.

5.2 Which Graph-aware Paradigm is More Susceptible to Graph Adversarial Attacks and Why?

Table 2 shows that LLAGA is more susceptible to structural attacks than GRAPHPROMPTER, a consequence of its heightened *sensitivity to structural perturbations*. LLAGA encodes graph structure through two coupled mechanisms: a fixed-length node-level sequence generated per node and a graph Laplacian embedding, where each sequence position maps deterministically to a relative structural position in the original graph. Because these node embeddings are computed directly from the perturbed adjacency structure, structural perturbations propagate through to the token stream consumed by the LLM and thereby degrade accuracy. In GRAPHPROMPTER, by contrast, only the GNN branch is exposed to structural perturbation while the textual node embeddings are unaffected; the intact textual features can then compensate for the corrupted structure, which accounts for its greater robustness.

6 Node Sequence Template Injection Attacks

We characterize a new attack surface in LLAGA that arises from the neighborhood detail template used to construct its node sequences. The attack is illustrated in Figure 2 and formalized in the algorithm of Appendix A.4. To construct the node sequence for a node u , the neighborhood detail template builds a *fixed-shape* computational tree in which the children of each node are k sampled 1-hop neighbors, applied recursively at every level. When a node has fewer than k neighbors, the remaining child slots are filled with placeholder tokens to preserve the fixed arity (e.g., $k = 10$), as prescribed by LLAGA’s original design (see the “Clean” panel of Figure 2 and lines 1–14 of the algorithm). This deterministic placeholder-padding rule is precisely what exposes the attack surface.

Exploiting the neighborhood detail template, an adversary can insert malicious nodes into the placeholder slots via graph manipulation and thereby degrade predictive accuracy. We formalize three injection strategies. The first, *Non-Adjacent*

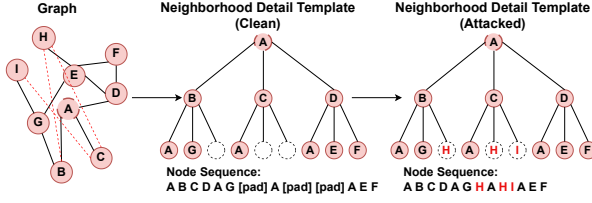


Figure 2: Schematic of the node sequence template injection attacks, depicted for the Non-adjacent Injection (NI) variant. Red dashed lines denote the edges inserted by the adversary. Under the Supernode Injection (SI) and Multiple Supernode Injection (MSI) variants, one or more supernodes are inserted at the placeholder positions. The arity is $k = 3$ in this illustration.

Injection (NI), inserts nodes drawn from outside the 2-hop neighborhood of the target node u (see the "Attacked" panel of Figure 2 and lines 21–27). The second, *Supernode Injection (SI)*, inserts a single high-degree node—again constrained to lie outside the target’s 2-hop neighborhood—into a placeholder slot. The third, *Multiple Supernode Injection (MSI)*, generalizes SI by inserting several distinct supernodes across the available placeholder slots (lines 35–43). The underlying mechanism is that populating the node sequence with uninformative nodes corrupts the target node’s aggregated representation, which in turn yields a poorly optimized projection layer and degrades both the training and inference behavior of the model.

Results. As reported in Table 3, LLAGA remains highly resilient to the poisoning variant on Cora and Citeseer but degrades sharply under evasion: accuracy falls to 0.42, 0.36, and 0.30 on Cora and to 0.36, 0.30, and 0.24 on Citeseer for the NI, SI, and MSI variants, respectively.

On the larger PubMed graph, by contrast, the model is substantially more robust to all three injection variants, exhibiting only a modest evasion-time accuracy reduction (to 0.85, 0.82, and 0.76). We attribute this to a scale-dependent locality effect: on the smaller, more tightly connected Cora and Citeseer graphs a localized injection concentrates its impact and severely disrupts local-neighborhood reasoning, whereas on the much larger PubMed graph the same perturbation is diluted across the neighborhood and its effect on accuracy is correspondingly attenuated.

The injection attacks are more effective than the transferred structural baselines. For example, under evasion the MSI variant drives accuracy to 0.30, 0.24, and 0.76 on Cora, CiteSeer, and PubMed, whereas METAATTACK reaches only 0.44, 0.50, and 0.73 on the same datasets (Table 2).

7 Poisoning and Evasion Attack via Feature Perturbation

Prior graph adversarial attacks operate almost exclusively on the graph structure (Li et al. 2023; Wu et al. 2019; Dai et al. 2018; Sun et al. 2022; Zügner and Günnemann 2019); the sole feature-oriented exception, NETTACK (Zügner, Akbarnejad, and Günnemann 2018), does not transfer to the

Table 3: Node-classification accuracy under the node sequence template injection attack across datasets and injection variants. NI, SI, and MSI denote the non-adjacent, single-supernode, and multiple-supernode injection variants, respectively; *Clean* denotes accuracy in the absence of any attack.

Data	Attack	NI	SI	MSI
Cora	Clean	—	0.89 ± 0.07	—
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	—	0.71 ± 0.04	—
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	—	0.90 ± 0.04	—
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03

textual feature domain. Because structural attacks are only moderately effective against robust models such as GRAPH-PROMPTER, and because textual node features are a dominant determinant of accuracy, we investigate imperceptible feature perturbations as an alternative attack vector.

We adapt the imperceptible-perturbation strategies of (Boucher et al. 2022) with one modification: we drive the perturbation with a task-agnostic English-to-French translation objective. The attack seeks input-text modifications that are visually indistinguishable to a human reader yet induce an incorrect translation, on the premise that a corrupted translation is a proxy for corrupted encoded semantics. We instantiate two perturbation families: *Homoglyphs*, which substitute characters with visually equivalent Unicode code points, and *Reorderings*, which insert bidirectional control characters to alter the rendered character order. Worked examples of both appear in Appendix A.6.

Following (Boucher et al. 2022), we optimize the perturbations with differential evolution (Storn and Price 1997), maximizing the Levenshtein distance between the model outputs on the perturbed and unperturbed text. The perturbation budget is fixed at 10% of the mean length of each textual feature.

Results. Table 4 shows that Reordering perturbations are uniformly more potent than Homoglyph perturbations across all models and datasets. On Cora and CiteSeer, LLAGA is vulnerable to both families: under poisoning, Homoglyph reduces accuracy by 27 and 23 percentage points, and Reordering by a larger 35 and 25 percentage points, respectively. GRAPH-PROMPTER is more resilient in both cases. PubMed follows the same qualitative pattern, with smaller absolute effects than on the two smaller datasets.

Contrasting feature and structural perturbations reveals a qualitative shift in the dominant vulnerability (Figure 3). Feature perturbations—Reordering in particular—outperform the structural attack (METAATTACK) by a wide margin on both LLAGA and GRAPH-PROMPTER, indicating that graph-aware LLMs weight the input features more heavily than the graph structure and are therefore more exposed to feature-level perturbation. We examine the interaction of the two

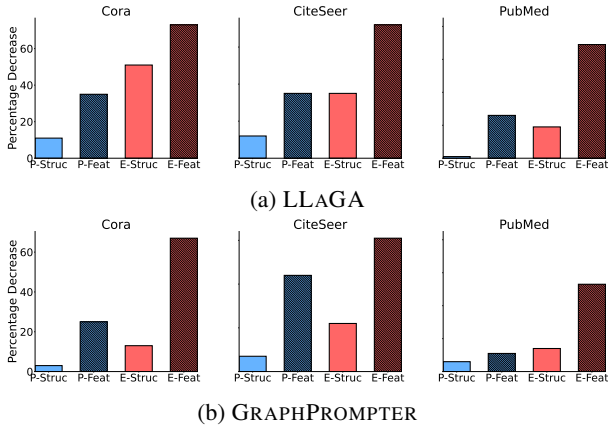


Figure 3: Comparison of structure-based and feature-based adversarial attacks on graph-aware LLMs. Attack success is quantified as the percentage decrease in accuracy relative to the clean model (y-axis). The labels P-Struc, P-Feat, E-Struc, and E-Feat denote, respectively, the poisoning and evasion attacks instantiated with structural and with feature perturbations.

attack modalities under a unified setting in the next section.

8 A Unified Imperceptible Attack on Graph-aware LLMs

We propose a unified attack that jointly applies feature and structural perturbations to yield a more effective yet still imperceptible adversary. We hypothesize that combining the two modalities degrades accuracy more than either applied alone, while keeping both the structural and feature modifications below the perceptibility threshold.

The structural component uses METAATTACK to perturb the edge set, and the feature component applies the combined Homoglyph and Reordering perturbations of the preceding section. Jointly, these perturbations constitute a low-visibility attack that simultaneously corrupts the relational and textual signals on which graph-aware LLMs depend.

Results. Table 5 shows that the unified attack, which applies structural and feature perturbations jointly, degrades accuracy substantially more than either modality in isolation across all datasets. This result isolates a key architectural property of graph-aware LLMs: unlike conventional GNNs, whose vulnerability is dominated by structural perturbations, these models are additionally and substantially exposed to feature-level perturbations. The efficacy of the unified attack indicates that they depend on the mutual consistency of graph structure and node features, rendering them highly susceptible to coordinated, low-visibility modifications of both.

9 Ablation Study

9.1 Varying Feature Perturbation

To quantify the dependence of attack success on perturbation magnitude, we vary the fraction of perturbed features

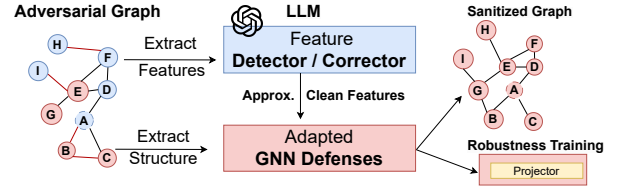


Figure 4: **Overview of the GALGUARD defense.** Perturbed node features and perturbed edges are colored blue and red, respectively. The LLM detects perturbations in the node features extracted from an adversarial graph and emits a corrected version. The corrected features, together with the perturbed edge set, are then processed by existing GNN defenses, which are used either to produce a sanitized graph or, in adapted form, to drive robustness training of the projector.

over the range 1% to 10%. We analyze the Reordering attack, whose behavior mirrors that of the Homoglyph attack. Across all models and datasets (Cora, Citeseer, and PubMed), decreasing the perturbed-feature fraction monotonically reduces attack success, as expected. Notably, as the perturbation magnitude decreases, the poisoning and evasion success rates converge: at low budgets the injected adversarial signal is too weak to materially affect the model, irrespective of whether it is applied during training (poisoning) or inference (evasion). The corresponding trend plot is provided in the Appendix (Figure 2).

9.2 Does the Choice of LLM Affect Attack Performance?

We test whether the choice of backbone LLM affects the adversarial robustness of graph-aware LLMs. In their default configurations, LLAGA uses Vicuna-7B (Chiang et al. 2023) and GRAPH_PROMPTER uses Llama2-7B (Touvron et al. 2023). We substitute Llama2-7B for LLAGA’s backbone and compare accuracy under poisoning and evasion attacks across all datasets. The two backbones yield comparable clean accuracy (e.g., on Cora, Vicuna: 0.89 ± 0.07 ; Llama2: 0.90 ± 0.03) and comparable degradation under attack, and the same holds when the backbones are swapped in GRAPH_PROMPTER. We therefore conclude that the choice of backbone LLM has negligible effect on adversarial robustness. Full results are reported in Table 2 of the Appendix.

10 Defense: GALGUARD

Existing GNN defenses are inadequate for graph-aware LLMs because these models are jointly vulnerable to structural and feature perturbations. A large class of GNN defenses prune or reweight edges according to node-feature similarity (Wu et al. 2019; Dai et al. 2018; Zhang and Zitnik 2020; Jin et al. 2020; Zhang and Ma 2024), and thus presuppose access to clean features; when the features are themselves perturbed, these methods fail (see Figure 3 in the Appendix). Likewise, defenses that decouple structure from features (Shanthamallu, Thiagarajan, and Spanias 2021; Wu et al. 2021), as well as OCR-based text sanitization (Boucher et al. 2022), break down under a coordinated attack on both

Table 4: Node-classification accuracy under feature perturbation attacks; parenthesized values give the absolute percentage-point drop from clean accuracy.

Data	Method	Poisoning		Evasion	
		LLAGA	GRAPHPROMPTER	LLAGA	GRAPHPROMPTER
Cora	Homoglyph	0.65 $\pm$ 0.05 (27%)	0.52 $\pm$ 0.04 (13%)	0.41 $\pm$ 0.04 (54%)	0.38 $\pm$ 0.03 (37%)
	Reordering	0.58 $\pm$ 0.06 (35%)	0.45 $\pm$ 0.05 (25%)	0.24 $\pm$ 0.05 (73%)	0.20 $\pm$ 0.03 (67%)
Citeseer	Homoglyph	0.54 $\pm$ 0.04 (23%)	0.65 $\pm$ 0.04 (7%)	0.32 $\pm$ 0.03 (55%)	0.52 $\pm$ 0.05 (26%)
	Reordering	0.46 $\pm$ 0.03 (35%)	0.39 $\pm$ 0.02 (44%)	0.20 $\pm$ 0.05 (72%)	0.27 $\pm$ 0.02 (61%)
PubMed	Homoglyph	0.72 $\pm$ 0.04 (20%)	0.83 $\pm$ 0.04 (8%)	0.36 $\pm$ 0.06 (60%)	0.55 $\pm$ 0.03 (39%)
	Reordering	0.67 $\pm$ 0.05 (26%)	0.80 $\pm$ 0.03 (11%)	0.28 $\pm$ 0.04 (69%)	0.42 $\pm$ 0.04 (53%)

modalities. This joint vulnerability motivates an integrated defense.

To this end, we propose the graph-aware LLM defense (GALGUARD), an end-to-end strategy that composes two complementary components: (1) an LLM-based feature corrector and (2) adapted GNN-based structural defenses.

Feature Robustness Strategy. As depicted in Figure 4, an LLM acts as an adaptive detector that scans the textual node features for anomalous patterns and emits approximate corrections that restore feature integrity, thereby attenuating feature-level perturbations. This correction step provides a practical countermeasure to the feature-level vulnerability. The exact instructions and prompt are given in Appendix A.6. In our experiments the corrector is instantiated with GPT-4 Turbo.

Graph Structure Robustness Strategy. Structural robustness is achieved via a two-stage procedure. First, as a pre-processing step, we apply a *graph purification heuristic* (Wu et al. 2019): for each node v we compute the cosine similarity between its embedding $\phi(x_v)$ and each neighbor embedding $\phi(x_u)$, and delete edges whose similarity falls below threshold. Second, we integrate *GNNGuard* (Zhang and Zitnik 2020). For GRAPHPROMPTER, GNNGuard is applied directly to the message-passing layers. For LLAGA, we introduce a novel adaptation: the purification step is folded into the construction of the computational tree used to build the node sequence, and we augment LLAGA’s projector with a learnable *global structural context embedding* M_{global} . Concatenating M_{global} with the purified node-sequence embedding and training the two jointly enables the model to operate robustly on potentially perturbed graph inputs. The modular design of GALGUARD is plug-and-play; alternative structural defenses—for example, Pro-GNN (Jin et al. 2020)—may be substituted for GNNGuard without further modification.

10.1 Results

Table 6 reports node-classification accuracy under METAATTACK for the undefended model and for two configurations of our defense: GALGUARD_p, which composes the LLM feature corrector with the graph purification module, and the full GALGUARD, which additionally incorporates GNNGuard (or its LLAGA adaptation). Across all datasets (Cora,

Table 5: Node-classification accuracy under the unified imperceptible attack; parenthesized values give the absolute percentage-point drop from clean accuracy.

Data	Attack	LLAGA	GRAPHPROMPTER
Cora	Poisoning	0.35 $\pm$ 0.05 (61%)	0.28 $\pm$ 0.08 (53%)
	Evasion	0.14 $\pm$ 0.06 (84%)	0.11 $\pm$ 0.05 (82%)
Citeseer	Poisoning	0.26 $\pm$ 0.04 (63%)	0.22 $\pm$ 0.03 (69%)
	Evasion	0.10 $\pm$ 0.09 (86%)	0.13 $\pm$ 0.07 (81%)
PubMed	Poisoning	0.39 $\pm$ 0.07 (57%)	0.32 $\pm$ 0.04 (64%)
	Evasion	0.12 $\pm$ 0.05 (87%)	0.19 $\pm$ 0.04 (79%)

CiteSeer, and PubMed) and both attack modalities, the defenses consistently recover accuracy lost to METAATTACK. GALGUARD_p already provides a strong baseline, confirming that correcting feature perturbations with the LLM and applying initial graph purification jointly improve robustness. The full GALGUARD consistently improves further over GALGUARD_p, which we attribute to the complementary contribution of the integrated GNN-based structural defense.

Table 6: Node-classification accuracy under METAATTACK and under the proposed defense GALGUARD, for the poisoning and evasion modalities. GALGUARD_p denotes the configuration that composes feature correction with graph purification.

Data	Method	LLAGA		GRAPHPROMPTER	
		Poisoning	Evasion	Poisoning	Evasion
Cora	METAATTACK	0.79 $\pm$ 0.03	0.44 $\pm$ 0.06	0.58 $\pm$ 0.05	0.52 $\pm$ 0.04
	GALGUARD _p	0.82 $\pm$ 0.05	0.62 $\pm$ 0.04	0.59 $\pm$ 0.03	0.56 $\pm$ 0.04
	GALGUARD	0.85$\pm$0.04	0.83$\pm$0.04	0.60$\pm$0.05	0.59$\pm$0.03
CiteSeer	MetaAttack	0.63 $\pm$ 0.04	0.50 $\pm$ 0.03	0.65 $\pm$ 0.06	0.56 $\pm$ 0.03
	GALGUARD _p	0.64 $\pm$ 0.03	0.56 $\pm$ 0.02	0.67 $\pm$ 0.05	0.60 $\pm$ 0.06
	GALGUARD	0.67$\pm$0.04	0.62$\pm$0.04	0.68$\pm$0.04	0.64$\pm$0.05
PubMed	MetaAttack	0.89 $\pm$ 0.08	0.73 $\pm$ 0.04	0.85 $\pm$ 0.02	0.77 $\pm$ 0.03
	GALGUARD _p	0.90 $\pm$ 0.06	0.78 $\pm$ 0.03	0.85 $\pm$ 0.04	0.81 $\pm$ 0.04
	GALGUARD	0.90$\pm$0.01	0.87$\pm$0.02	0.89$\pm$0.02	0.89$\pm$0.01

11 Conclusion

This paper provides the first systematic characterization of the adversarial robustness of graph-aware LLMs, a previously unexamined problem. Our analysis establishes that the graph-integration mechanism is itself a principal source of vulnerability: we identify new attack surfaces and show that design choices such as LLAGA’s node sequence template measurably increase susceptibility to attack. We further demonstrate that—in contrast to conventional GNNs—feature perturbation attacks are highly effective and frequently exceed the impact of structural attacks, and that a unified attack combining both perturbation modalities reduces graph-aware LLMs to near-random accuracy.

In response, we propose GALGUARD, an end-to-end defense that integrates an LLM-based feature corrector with adapted GNN-based structural defenses. As graph-aware LLMs are increasingly deployed in domains such as social networks, healthcare, and finance, guaranteeing their resilience to adversarial manipulation is essential. We intend this work to serve as a foundation for subsequent research into the vulnerabilities and defenses of graph-aware LLMs, and to inform both the theoretical understanding and the practical safeguards required for their real-world deployment.

References

- Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F. L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Ahmedt-Aristizabal, D.; Armin, M. A.; Denman, S.; Fookes, C.; and Petersson, L. 2021. Graph-based deep learning for medical diagnosis and analysis: past, present and future. *Sensors*, 21(14): 4758.
- Anil, R.; Dai, A. M.; Firat, O.; Johnson, M.; Lepikhin, D.; Passos, A.; Shakeri, S.; Taropa, E.; Bailey, P.; Chen, Z.; et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403*.
- Bi, B.; Liu, S.; Wang, Y.; Mei, L.; and Chen, X. 2024. Scalable Link Prediction on Large-Scale Heterogeneous Graphs with Large Language Models. *arXiv preprint arXiv:2401.13227*.
- Boucher, N.; Shumailov, I.; Anderson, R.; and Papernot, N. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*, 1987–2004. IEEE.
- Brannon, W.; Fulay, S.; Jiang, H.; Kang, W.; Roy, B.; Kabbara, J.; and Roy, D. 2023. Congrat: Self-supervised contrastive pretraining for joint graph and text embeddings. *arXiv preprint arXiv:2305.14321*.
- Chai, Z.; Zhang, T.; Wu, L.; Han, K.; Hu, X.; Huang, X.; and Yang, Y. 2023. GraphLLM: Boosting Graph Reasoning Ability of Large Language Model. *arXiv preprint arXiv:2310.05845*.
- Chandra, S.; Mishra, P.; Yannakoudakis, H.; Nimishakavi, M.; Saeidi, M.; and Shutova, E. 2020. Graph-based Modeling of Online Communities for Fake News Detection. *arXiv preprint arXiv:2008.06274*.
- Chen, R.; Zhao, T.; Jaiswal, A.; Shah, N.; and Wang, Z. 2024. LLaGA: Large Language and Graph Assistant. In *Forty-first International Conference on Machine Learning*.
- Chen, Z.; Mao, H.; Li, H.; Jin, W.; Wen, H.; Wei, X.; Wang, S.; Yin, D.; Fan, W.; Liu, H.; and Tang, J. 2023a. Exploring the Potential of Large Language Models (LLMs) in Learning on Graphs. *arXiv preprint arXiv:2307.03393*.
- Chen, Z.; Mao, H.; Wen, H.; Han, H.; Jin, W.; Zhang, H.; Liu, H.; and Tang, J. 2023b. Label-free Node Classification on Graphs with Large Language Models (LLMs). *arXiv preprint arXiv:2310.04668*.
- Chiang, W.-L.; Li, Z.; Lin, Z.; Sheng, Y.; Wu, Z.; Zhang, H.; Zheng, L.; Zhuang, S.; Zhuang, Y.; Gonzalez, J. E.; Stoica, I.; and Xing, E. P. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.
- Conti, M.; Li, J.; Picek, S.; and Xu, J. 2022. Label-only membership inference attack against node-level graph neural networks. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, 1–12.
- Dai, E.; Lin, M.; Zhang, X.; and Wang, S. 2023. Unnoticeable backdoor attacks on graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2263–2273.
- Dai, H.; Li, H.; Tian, T.; Huang, X.; Wang, L.; Zhu, J.; and Song, L. 2018. Adversarial attack on graph structured data. In *International conference on machine learning*, 1115–1124. PMLR.
- Dong, N.; Mücke, S.; and Khosla, M. 2022. Mucomid: A multitask graph convolutional learning framework for miRNA-disease association prediction. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6): 3081–3092.
- Duan, K.; Liu, Q.; Chua, T.-S.; Yan, S.; Ooi, W. T.; Xie, Q.; and He, J. 2023. SimTeG: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv preprint arXiv:2308.02565*.
- Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Duddu, V.; Boutet, A.; and Shejwalkar, V. 2020. Quantifying privacy leakage in graph embedding. In *MobiQuitous 2020-17th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services*, 76–85.
- Fan, W.; Ma, Y.; Li, Q.; He, Y.; Zhao, E.; Tang, J.; and Yin, D. 2019. Graph neural networks for social recommendation. In *The world wide web conference*, 417–426.
- Fatemi, B.; Halcrow, J.; and Perozzi, B. 2023. Talk like a Graph: Encoding Graphs for Large Language Models. *arXiv preprint arXiv:2310.04560*.
- Gong, N. Z.; and Liu, B. 2018. Attribute inference attacks in online social networks. *ACM Transactions on Privacy and Security (TOPS)*, 21(1): 1–30.
- Guo, J.; Du, L.; Liu, H.; Zhou, M.; He, X.; and Han, S. 2023. GPT4Graph: Can Large Language Models Understand Graph Structured Data ? An Empirical Evaluation and Benchmarking. *arXiv preprint arXiv:2305.15066*.
- Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30.
- He, X.; Bresson, X.; Laurent, T.; Perold, A.; LeCun, Y.; and Hooi, B. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *The Twelfth International Conference on Learning Representations*.
- He, X.; Wen, R.; Wu, Y.; Backes, M.; Shen, Y.; and Zhang, Y. 2021. Node-level membership inference attacks against graph neural networks. *arXiv preprint arXiv:2102.05429*.

- Hu, W.; Fey, M.; Zitnik, M.; Dong, Y.; Ren, H.; Liu, B.; Catasta, M.; and Leskovec, J. 2020. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 33: 22118–22133.
- Huang, J.; Zhang, X.; Mei, Q.; and Ma, J. 2023a. Can LLMs Effectively Leverage Structural Information for Graph Learning: When and Why. *arXiv preprint arXiv:2309.16595*.
- Huang, X.; Han, K.; Bao, D.; Tao, Q.; Zhang, Z.; Yang, Y.; and Zhu, Q. 2023b. Prompt-based Node Feature Extractor for Few-shot Learning on Text-Attributed Graphs. *arXiv preprint arXiv:2309.02848*.
- Jin, W.; Li, Y.; Xu, H.; Wang, Y.; Ji, S.; Aggarwal, C.; and Tang, J. 2021. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 22(2): 19–34.
- Jin, W.; Ma, Y.; Liu, X.; Tang, X.; Wang, S.; and Tang, J. 2020. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 66–74.
- Kipf, T. N.; and Welling, M. 2022. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.
- Kuang, H.; Xu, J.; Zhang, H.; Zhao, Z.; Zhang, Q.; Huang, X.; and Wei, Z. 2023. Unleashing the Power of Language Models in Text-Attributed Graph. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 8429–8441. Association for Computational Linguistics.
- Li, K.; Liu, Y.; Ao, X.; and He, Q. 2023. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*.
- Liu, C.; and Wu, B. 2023. Evaluating Large Language Models on Graphs: Performance Insights and Comparative Analysis. *arXiv preprint arXiv:2308.11224*.
- Liu, S.; Nie, W.; Wang, C.; Lu, J.; Qiao, Z.; Liu, L.; Tang, J.; Xiao, C.; and Anandkumar, A. 2023. Multi-modal molecule structure–text model for text-based retrieval and editing. *Nature Machine Intelligence*, 5(12): 1447–1457.
- Liu, Z.; He, X.; Tian, Y.; and Chawla, N. V. 2024. Can we soft prompt LLMs for graph learning tasks? In *Companion Proceedings of the ACM on Web Conference 2024*, 481–484.
- Ma, J.; Ding, S.; and Mei, Q. 2020. Towards more practical adversarial attacks on graph neural networks. *Advances in neural information processing systems*, 33: 4756–4766.
- Mavromatis, C.; Ioannidis, V. N.; Wang, S.; Zheng, D.; Adeshina, S.; Ma, J.; Zhao, H.; Faloutsos, C.; and Karypis, G. 2023. Train your own GNN teacher: Graph-aware distillation on textual graphs. *arXiv preprint arXiv:2304.10668*.
- Olatunji, I. E.; Hizber, A.; Sihlovec, O.; and Khosla, M. 2023a. Does black-box attribute inference attacks on graph neural networks constitute privacy risk? *arXiv preprint arXiv:2306.00578*.
- Olatunji, I. E.; Nejd, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *2021 Third IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 11–20. IEEE.
- Olatunji, I. E.; Rathee, M.; Funke, T.; and Khosla, M. 2023b. Private Graph Extraction via Feature Explanations. *Proceedings on Privacy Enhancing Technologies*.
- Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; and Wu, X. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*.
- Peng, Y.; Lin, S.; Chen, Q.; Xu, L.; Ren, X.; Li, Y.; and Xu, J. 2024. ChatGraph: Chat with Your Graphs. *arXiv preprint arXiv:2401.12672*.
- Qin, Y.; Wang, X.; Zhang, Z.; and Zhu, W. 2023. Disentangled Representation Learning with Large Language Models for Text-Attributed Graphs. *arXiv preprint arXiv:2310.18152*.
- Shanthamallu, U. S.; Thiagarajan, J. J.; and Spanias, A. 2021. Uncertainty-matching graph neural networks to defend against poisoning attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, 9524–9532.
- Storn, R.; and Price, K. 1997. Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *Journal of global optimization*, 11: 341–359.
- Su, B.; Du, D.; Yang, Z.; Zhou, Y.; Li, J.; Rao, A.; Sun, H.; Lu, Z.; and Wen, J.-R. 2022. A molecular multimodal foundation model associating molecule graphs with natural language. *arXiv preprint arXiv:2209.05481*.
- Sun, L.; Dou, Y.; Yang, C.; Zhang, K.; Wang, J.; Philip, S. Y.; He, L.; and Li, B. 2022. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(8): 7693–7711.
- Sun, S.; Ren, Y.; Ma, C.; and Zhang, X. 2023. Large Language Models as Topological Structure Enhancers for Text-Attributed Graphs. *arXiv preprint arXiv:2311.14324*.
- Tan, Y.; Zhou, Z.; Lv, H.; Liu, W.; and Yang, C. 2023. WalkLM: A Uniform Language Model Fine-tuning Framework for Attributed Graph Embedding. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Team, G.; Anil, R.; Borgeaud, S.; Wu, Y.; Alayrac, J.-B.; Yu, J.; Soricut, R.; Schalkwyk, J.; Dai, A. M.; Hauth, A.; et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.
- Tian, Y.; Song, H.; Wang, Z.; Wang, H.; Hu, Z.; Wang, F.; Chawla, N. V.; and Xu, P. 2024. Graph neural prompting with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 19080–19088.
- Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Liò, P.; and Bengio, Y. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.
- Wang, J.; Zhang, S.; Xiao, Y.; and Song, R. 2021. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*.
- Wang, X.; and Wang, W. H. 2022. Group property inference attacks against graph neural networks. In *Proceedings of the*

- 2022 ACM SIGSAC Conference on Computer and Communications Security, 2871–2884.
- Wei, Y.; Fu, S.; Jiang, W.; Kwok, J. T.; and Zhang, Y. 2024a. Rendering graphs for graph reasoning in multimodal large language models. *arXiv preprint arXiv:2402.02130*.
- Wei, Y.; Huang, Q.; Kwok, J. T.; and Zhang, Y. 2024b. Kicgpt: Large language model with knowledge in context for knowledge graph completion. *arXiv preprint arXiv:2402.02389*.
- Wenkel, F.; Wolf, G.; and Knyazev, B. 2023. Pretrained Language Models to Solve Graph Tasks in Natural Language. In *ICML workshop on Structured Probabilistic Inference & Generative Modeling*.
- Wu, H.; Wang, C.; Tyshetskiy, Y.; Docherty, A.; Lu, K.; and Zhu, L. 2019. Adversarial examples on graph data: Deep insights into attack and defense. *arXiv preprint arXiv:1903.01610*.
- Wu, X.; Wu, H.; Zhou, X.; and Lu, K. 2021. CoG: a Two-View Co-training Framework for Defending Adversarial Attacks on Graph. *arXiv preprint arXiv:2109.05558*.
- Xie, H.; Zheng, D.; Ma, J.; Zhang, H.; Ioannidis, V. N.; Song, X.; Ping, Q.; Wang, S.; Yang, C.; Xu, Y.; et al. 2023. Graph-aware language model pre-training on a large graph corpus can help multiple graph applications. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5270–5281.
- Yang, J.; Liu, Z.; Xiao, S.; Li, C.; Lian, D.; Agrawal, S.; Singh, A.; Sun, G.; and Xie, X. 2021. GraphFormers: GNN-nested Transformers for Representation Learning on Textual graph. In *Proceedings of the 35th Conference on Neural Information Processing Systems*.
- Yang, Z.; Cohen, W.; and Salakhudinov, R. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*, 40–48. PMLR.
- Ye, R.; Zhang, C.; Wang, R.; Xu, S.; and Zhang, Y. 2023. Natural Language is All a Graph Needs. *arXiv preprint arXiv:2308.07134*.
- Zhang, A.; and Ma, J. 2024. Defensevgae: Defending against adversarial attacks on graph data via a variational graph autoencoder. In *International Conference on Intelligent Computing*, 313–324. Springer.
- Zhang, J. 2023. Graph-toolformer: To empower llms with graph reasoning ability via prompt augmented by chatgpt. *arXiv preprint arXiv:2304.11116*.
- Zhang, X.; and Zitnik, M. 2020. Gnnguard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 33: 9263–9275.
- Zhang, Z.; Chen, M.; Backes, M.; Shen, Y.; and Zhang, Y. 2022. Inference attacks against graph neural networks. In *31st USENIX Security Symposium (USENIX Security 22)*, 4543–4560.
- Zhang, Z.; Liu, Q.; Huang, Z.; Wang, H.; Lu, C.; Liu, C.; and Chen, E. 2021. Graphmi: Extracting private graph data from graph neural networks. *arXiv preprint arXiv:2106.02820*.
- Zhang, Z.; Wang, X.; Zhang, Z.; Li, H.; Qin, Y.; and Zhu, W. 2023. LLM4DyG: Can Large Language Models Solve Problems on Dynamic Graphs? *arXiv preprint arXiv:2310.17110*.
- Zhao, J.; Zhuo, L.; Shen, Y.; Qu, M.; Liu, K.; Bronstein, M.; Zhu, Z.; and Tang, J. 2023. GraphText: Graph Reasoning in Text Space. *arXiv preprint arXiv:2310.01089*.
- Zhu, Y.; Wang, Y.; Shi, H.; and Tang, S. 2024. Efficient tuning and inference for large language models on textual graphs. *arXiv preprint arXiv:2401.15569*.
- Zügner, D.; Akbarnejad, A.; and Günnemann, S. 2018. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2847–2856.
- Zügner, D.; and Günnemann, S. 2019. Adversarial Attacks on Graph Neural Networks via Meta Learning. In *International Conference on Learning Representations (ICLR)*.

A Appendix

Organization: The Appendix is structured as follows. Appendix A.1 presents the detailed taxonomy of graph-aware LLMs, and Appendix A.2 reviews related work on attacks against Graph Neural Networks (GNNs). Appendix A.3 specifies the adversarial attack procedures NETTACK and METAATTACK, and Appendix A.4 formalizes the algorithm for the node sequence injection attack on LLAGA. Appendix A.5 reports the ablation studies, quantifying the effect of the perturbed-feature fraction and of the backbone LLM on attack effectiveness. Finally, Appendix A.7 provides worked examples of the feature attacks and the prompt template for the LLM corrector, and Appendix A.8 discusses the broader implications of our findings.

Code. Our code will be released upon acceptance.

A.1 Graph Encoding-Based Taxonomy of LLM Adaptations

The integration of large language models (LLMs) with graph-structured data is an emerging research direction, spanning a variety of mechanisms for adapting or augmenting LLMs to perform graph objectives such as reasoning, representation, and inference. We organize these mechanisms into two classes according to how the graph is encoded for the LLM: *textual descriptions of graphs*, in which graph content is serialized as natural language, and *learned graph projectors*, in which a trainable projector maps graph data into embeddings or structured token forms that the LLM can consume directly. This section describes representative methods in each class. Figure 5 illustrates the two schemes, and Table 7 tabulates their principal distinctions.

Graph Representation as a Textual Description In this first class, graph data is serialized into natural language so that the LLM processes the graph structure as a textual description. The class has been studied extensively: works such as (Guo et al. 2023; Liu and Wu 2023; Ye et al. 2023; Wenkel, Wolf, and Knyazev 2023; Wei et al. 2024b; Fatemi, Halcrow, and Perozzi 2023; Tan et al. 2023; Zhao et al. 2023; Wei et al. 2024a) serialize graph data into textual form, typically paired with task-specific query instructions. For example, Tan et al. (2023) proposed a unified attributed-graph embedding framework that generates random-walk-based textual sequences from the graph and fine-tunes an LLM on them. Zhao et al. (2023) addressed graph reasoning in text space by translating graph structures into natural language to enable LLM interaction, and Wei et al. (2024a) extended this by augmenting the textual graph representation with a rendered visual representation to strengthen LLM reasoning. In the multi-modal setting, Liu et al. (2023) and Su et al. (2022) aligned molecular graphs with textual descriptions to advance drug discovery and molecular retrieval. Huang et al. (2023a) examined whether injecting graph structure into LLM prompts improves predictive accuracy and reported that LLMs treat graph-related prompts as free-form contextual text rather than as explicit structural objects, which reinforces the sensitivity of these methods to prompt design—also exploited by (Brannon et al. 2023; Zhang et al. 2023; Sun et al. 2023;

Bi et al. 2024; Pan et al. 2024) for heterogeneous and large-scale graph objectives. Complementarily, (He et al. 2024) used LLM-generated explanations as features to improve both accuracy and interpretability on graphs.

A related line of work retains the textual-description encoding but couples it with GNN pretraining strategies to strengthen the LLM’s graph reasoning. Chen et al. (2023a) proposed two LLM–GNN integration schemes: (1) using the LLM to augment node features for node classification, and (2) using the LLM as a direct predictor for graph objectives. Chandra et al. (2020) employed the LLM as a text encoder and a GNN as a structure encoder for tasks such as fake-news detection in online communities. Chen et al. (2023b) performed label-free node classification by using the LLM to annotate nodes, then training a GNN on the resulting annotations, and Mavromatis et al. (2023) used the LLM as a feature extractor for a downstream GNN.

For the inference stage, Zhu et al. (2024) proposed a fine-tuning procedure for textual graphs that couples LLMs and GNNs through a tunable side network attached to each LLM layer, improving both training efficiency and inference throughput. Duan et al. (2023) introduced SimTeG, in which the LLM is first fine-tuned on a downstream task (e.g., node classification) and node embeddings are then obtained from the last hidden states of the fine-tuned model. Zhang (2023) proposed Graph-toolformer, which uses ChatGPT-driven prompt engineering to augment the LLM’s graph reasoning.

While these methods lower the barrier to applying LLMs on graph objectives, they tend to under-represent the higher-order structural dependencies inherent in graph data, and their accuracy remains strongly dependent on prompt engineering—a process that requires careful, empirical design to reach optimal performance.

Graph Representation as a Learned Projector To overcome the representational limits of textual serialization, learned-projector methods map graph data into embeddings or graph vectors that the LLM can process directly. Chen et al. (2024) introduced LLAGA, a large language and graph assistant that preserves the general-purpose behavior of the LLM while mapping graph structure into graph tokens suitable for LLM input. (Liu et al. 2024; Qin et al. 2023) injected graph-structure information into a pre-trained LLM through purpose-built disentangled graph neural network layers. Tian et al. (2024) encoded knowledge graphs with graph neural prompts, producing graph embeddings for use at inference time. Huang et al. (2023b) proposed a prompt-based node-feature extractor (G-Prompt) for few-shot learning on text-attributed graphs, coupling a graph adapter (a learnable GNN layer) with task-specific prompts to aggregate neighborhood information and extract informative node features.

A further line of work compresses graph content into fixed-length prefixes via graph transformers attached to each LLM layer to augment its reasoning (Chai et al. 2023). Yang et al. (2021) developed GraphFormers, which interleaves text encoding and graph aggregation in an iterative workflow by nesting GNNs within the transformer blocks of a language model. Peng et al. (2024) developed ChatGraph, an interac-

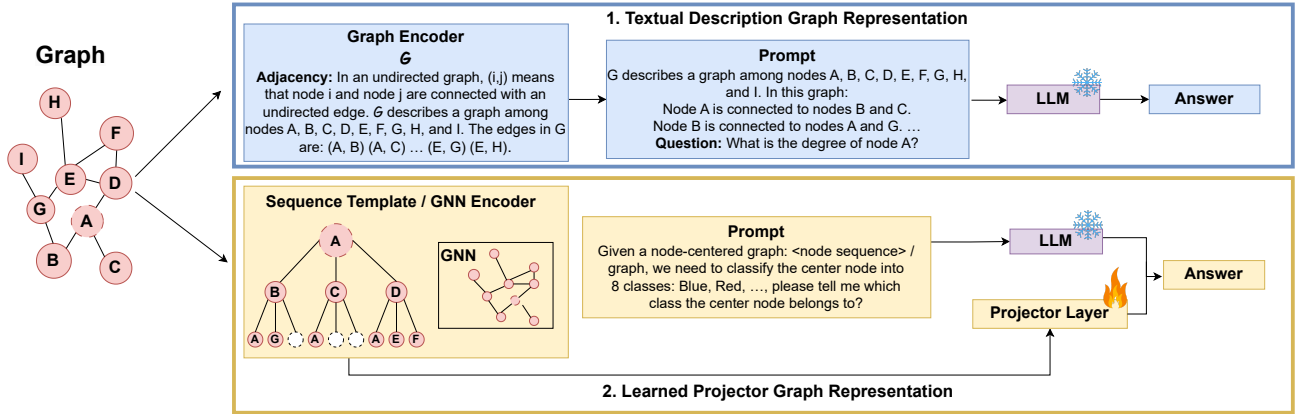


Figure 5: **Graph encoding-based adaptations of LLMs.** Under the textual-description scheme, the graph is serialized into natural language (top). Under the learned-projector scheme, the graph is encoded either by node templates that map it to token sequences or by a GNN (bottom). In both schemes the LLM parameters are frozen; only the projector that maps graph content to input tokens is optimized.

Table 7: Comparison of the two graph-encoding-based adaptation schemes for LLMs.

Aspect	Textual Description Approach	Learned Projector Approach
Graph Encoding	Serializes graph data (nodes, edges) into natural-language descriptions.	Maps graph structure into embeddings or graph tokens via a trainable projector.
LLM Modifications	LLM parameters unchanged.	LLM parameters unchanged.
Graph Structure Representation	Expresses graph topology and relations as textual narratives.	Encodes relational and structural information directly as vector representations.
Flexibility	Restricted to simple graphs.	Represents complex graph structures more flexibly.
Computational Cost	Requires no additional training; lower cost.	Trains a lightweight projector, typically a linear layer; minimal cost.
Suitability for Graph Tasks	May discard relational nuance owing to the limits of textual serialization.	Preserves fine-grained structural information, improving accuracy on graph-specific objectives.

tive natural-language interface to graph data in which graphs are represented as sequential paths and the LLM is fine-tuned on the graph to support graph analysis.

We exclude projector adaptations that require retraining the LLM, as they incur substantial computational cost without commensurate accuracy gains. We instead focus on adaptations built on **frozen pre-trained LLMs** (Chen et al. 2024; Liu et al. 2024; Qin et al. 2023; Tian et al. 2024; Huang et al. 2023b), which reduce computational cost, remain portable across LLM backbones, and exploit the LLM’s pre-trained knowledge without retraining overhead while still specializing to the target graph objective. Throughout, we adopt two representative projector methods: **LLAGA** (Chen et al. 2024), which pairs node-level templates with a linear projector, and **GRAPHPROMPTER** (Liu et al. 2024), which encodes the graph with a GNN-based projector.

This work. We exclude textual-description methods for two reasons. First, they under-represent the higher-order structural dependencies inherent in graph data: they are effective only for simple queries—for instance, reporting basic properties such as node degree—and scale poorly, since the prompt token limit bounds the level of detail expressible for larger graphs and is exceeded rapidly as graph size grows. Second, from a security standpoint, textual-description methods expose no graph-specific attack surface: because the graph is rendered as plain text (e.g., “Node A is connected to Node B”), they are vulnerable to conventional text-based adversarial attacks rather than to the structural or feature perturbations characteristic of graph-based models. They thus lack the structure that would admit the novel, graph-specific vulnerabilities central to our study of graph-aware LLM robustness.

A.2 Attacks on GNN

The attack literature on GNNs spans several distinct objectives: membership inference attacks (Olatunji, Nejdli, and Khosla 2021; He et al. 2021; Conti et al. 2022), which determine whether a given node belonged to the training set; attribute inference attacks (Gong and Liu 2018; Olatunji et al. 2023a; Duddu, Boutet, and Shejwalkar 2020), which recover sensitive or missing node attributes from model outputs and learned embeddings; property inference attacks (Zhang et al. 2022; Wang and Wang 2022), which extract global properties of the entire graph; adversarial attacks (Zügner, Akbarnejad, and Günnemann 2018; Zügner and Günnemann 2019; Li et al. 2023; Dai et al. 2018; Sun et al. 2022), which degrade predictive performance via bounded perturbations of the graph or node features; and graph reconstruction attacks (Zhang et al. 2021; Olatunji et al. 2023b), which recover the underlying graph structure from learned embeddings or model outputs.

Among these, adversarial attacks are of primary concern here because they act directly on predictive performance. Such attacks apply small, optimized *perturbations* to the graph—modifying node features or inserting/deleting edges—so as to maximize the degradation of GNN accuracy. They fall into two categories: **poisoning attacks**, which manipulate the data during the *training phase*, and **evasion attacks**, which perturb the input graph at *test time* to induce misprediction. Zügner, Akbarnejad, and Günnemann (2018)

showed that structural or feature perturbations can substantially degrade GNN accuracy; their method, **NETTACK**, uses greedy optimization to compute the minimal perturbation that maximizes the training loss, thereby identifying the smallest feature or structural change sufficient to flip a prediction. Li et al. (2023) re-examined graph adversarial attacks from a data-distribution perspective, showing that adversarial examples shift the underlying data distribution, and proposed a defense to counteract this effect. Wu et al. (2019) presented a comprehensive analysis of graph adversarial examples covering both attacks and defenses, characterizing GNN sensitivity to small perturbations and proposing detection-based defenses that improve robustness. **METAATTACK** (Zügner and Günnemann 2019) performs both poisoning and evasion attacks by treating the graph structure as an optimizable hyperparameter, modifying it directly by gradient descent under a bi-level optimization formulation. It relies on meta-gradients—the gradients of the post-training attacker loss with respect to structural perturbations—to select the edge modifications that most degrade performance.

In this work we use **NETTACK** (Zügner, Akbarnejad, and Günnemann 2018) and **METAATTACK** (Zügner and Günnemann 2019) as representative adversarial attacks covering both poisoning and evasion, where poisoning targets model integrity during training and evasion targets performance at inference time.

A.3 How NETTACK and METAATTACK Work

NETTACK perturbs the graph structure by inserting and deleting edges under a constraint that approximately preserves the degree distribution of the perturbed graph. It employs greedy optimization to compute the minimal perturbation that maximizes the training loss of a surrogate model, taken to be a linearized two-layer graph convolutional network (GCN) that approximates the dependence of the classification output on the graph structure. The attack objective is to increase the model’s confidence in incorrect labels by maximizing the divergence between the logits on the perturbed and clean graphs. Operationally, **NETTACK** enumerates candidate edges around a target node and evaluates, for each incident edge, a score quantifying the effect of modifying that edge on the target node’s classification; it then selects the highest-scoring edge and either inserts or deletes it. Because **NETTACK** is a targeted attack (it induces misclassification of a single specified node), we follow (Zügner and Günnemann 2019) to obtain an untargeted evaluation: we attack each sampled test node with **NETTACK** while accounting for all nodes in the graph, and aggregate the per-node outcomes over the test set.

METAATTACK optimizes the graph structure directly by gradient descent, treating the adjacency matrix as a hyperparameter, and casts the attack as a bi-level optimization problem in which the inner level minimizes the training loss on the clean graph and the outer level minimizes the attacker loss on the poisoned graph. It uses the same linearized two-layer GCN surrogate to approximate the effect of structural changes on performance. A score function ranks candidate perturbations, and the attack uses the sign of the meta-gradients (gradients with respect to the hyperparameter) at connected node pairs to indicate which edges to modify; it

then greedily applies the highest-scoring edge insertions or deletions to maximize the attack’s impact.

For both procedures we adopt the unnoticeability constraint of (Zügner, Akbarnejad, and Günnemann 2018), which bounds the change in the graph’s degree distribution, and accordingly perturb at most 10% of the edges.

In the poisoning protocol for LLAGA, we take the perturbed graph produced by NETTACK or METAATTACK, re-generate the node-level sequences from it, and retrain the projector that maps these sequences to token embeddings. For GRAPHPROMPTER, we re-encode the perturbed graph with a graph attention network (GAT) and retrain its projector on the perturbed graph. In the evasion protocol, the graph-aware LLM is trained on the clean graph and the perturbation is applied only at inference time.

A.4 Algorithm for Node Sequence Template Injection Attacks

Algorithm 1 specifies the node sequence template injection attack on LLAGA. The model constructs a *fixed-shape* computational tree for each node u , sampling k neighbors per node up to depth d (lines 1–14); when a node has fewer than k neighbors, the remaining child slots are filled with placeholders, which constitutes the attack surface. We exploit this by injecting malicious nodes into the placeholder positions through graph manipulation, under three strategies: *Non-Adjacent Injection (NI)* inserts nodes drawn from outside the 2-hop neighborhood of u (lines 21–27); *Supernode Injection (SI)* inserts a single high-degree node into one placeholder (lines 29–33); and *Multiple Supernode Injection (MSI)* distributes distinct high-degree nodes across all available placeholders (lines 35–43). Populating the node sequence with these uninformative nodes corrupts the target node’s aggregated representation and thereby degrades the optimization of the projection layer.

A.5 Ablation Studies

This section reports the detailed results for the ablation studies summarized in the main paper.

Ablation Results: Varying Feature Perturbation Figure 6 plots the accuracy of the two graph-aware LLMs, LLAGA and GRAPHPROMPTER, as a function of the perturbed-feature fraction. For both models, increasing this fraction monotonically decreases accuracy, confirming their sensitivity to feature-level perturbations.

Ablation Results: Does the Choice of LLM Affect Attack Performance? Table 8 shows that, within the LLAGA framework, the choice of backbone LLM has negligible effect on both clean accuracy and adversarial vulnerability. Across Cora, Citeseer, and PubMed, the Vicuna-7B and Llama2-7B backbones yield near-identical clean accuracy and comparable degradation under both poisoning and evasion attacks. For example, under METAATTACK evasion on Cora, accuracy falls to 0.44 ± 0.06 with Vicuna-7B and 0.43 ± 0.08 with Llama2-7B—a difference well within one standard deviation. The same pattern holds across all datasets and attack types, indicating that attack effectiveness is essentially independent of the backbone LLM.

Algorithm 1: Injection Attacks on Neighborhood Detail Template for Obtaining Node Sequence.

Input: Graph $G = (V, E)$, target node u , tree depth d , children size k

Output: Modified graph G' with injected nodes

```

1: procedure NEIGHBORHOODDETAILTEMPLATE( $u, d, k$ )
2:    $T \leftarrow \emptyset$  ▷ Initialize empty tree
3:    $T.\text{root} \leftarrow u$ 
4:   for level  $\ell = 1$  to  $d$  do
5:     for each node  $v$  at level  $\ell - 1$  do
6:        $N_v \leftarrow \text{Sample}(\mathcal{N}(v), k)$  ▷ Sample  $k$  neighbors
7:       if  $|N_v| < k$  then
8:         Fill remaining slots with placeholders
9:       end if
10:      Add  $N_v$  as children of  $v$  in  $T$ 
11:    end for
12:  end for
13:  return  $T$ 
14: end procedure

15: procedure ATTACK( $G, u, d, k, \text{AttackStrategy}$ )
16:    $G' \leftarrow G$  ▷ Create copy of original graph
17:    $T \leftarrow \text{NEIGHBORHOODDETAILTEMPLATE}(u, d, k)$ 
18:    $P \leftarrow \text{GetPlaceholders}(T)$ 
19:   if  $|P| > 0$  then
20:      $\mathcal{N}_2 \leftarrow \text{2-hop neighborhood of } u$ 
21:     if AttackStrategy = NI then
22:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
23:        $V_{\text{inject}} \leftarrow \text{Sample}(V_{\text{non-adjacent}}, |P|)$ 
24:       for each placeholder position  $p \in P$  do
25:          $v \leftarrow \text{Next node from } V_{\text{inject}}$ 
26:         Add edge  $(\text{parent}(p), v)$  to  $G'$ 
27:       end for
28:     else if AttackStrategy = SI then
29:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
30:        $s \leftarrow \text{GetHighestDegreeNode}(V_{\text{non-adjacent}})$ 
31:       for first placeholder position  $p \in P$  do
32:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
33:         break ▷ Only use first placeholder
34:       end for
35:     else if AttackStrategy = MSI then
36:        $V_{\text{non-adjacent}} \leftarrow V \setminus \mathcal{N}_2$ 
37:        $S \leftarrow \text{GetNodesOrderedByDegree}(V_{\text{non-adjacent}})$ 
38:       for  $i = 1$  to  $|P|$  do
39:          $s \leftarrow \text{Get } i\text{-th node from } S$ 
40:          $p \leftarrow \text{Get } i\text{-th placeholder from } P$ 
41:         Add edge  $(\text{parent}(p), s)$  to  $G'$ 
42:       end for
43:     end if
44:   end if
45:   return  $G'$ 
46: end procedure

```

A.6 Results on the ArXiv Dataset

We now evaluate the injection attack on the large-scale ArXiv graph, comprising 169,343 nodes and 1,166,243 edges with

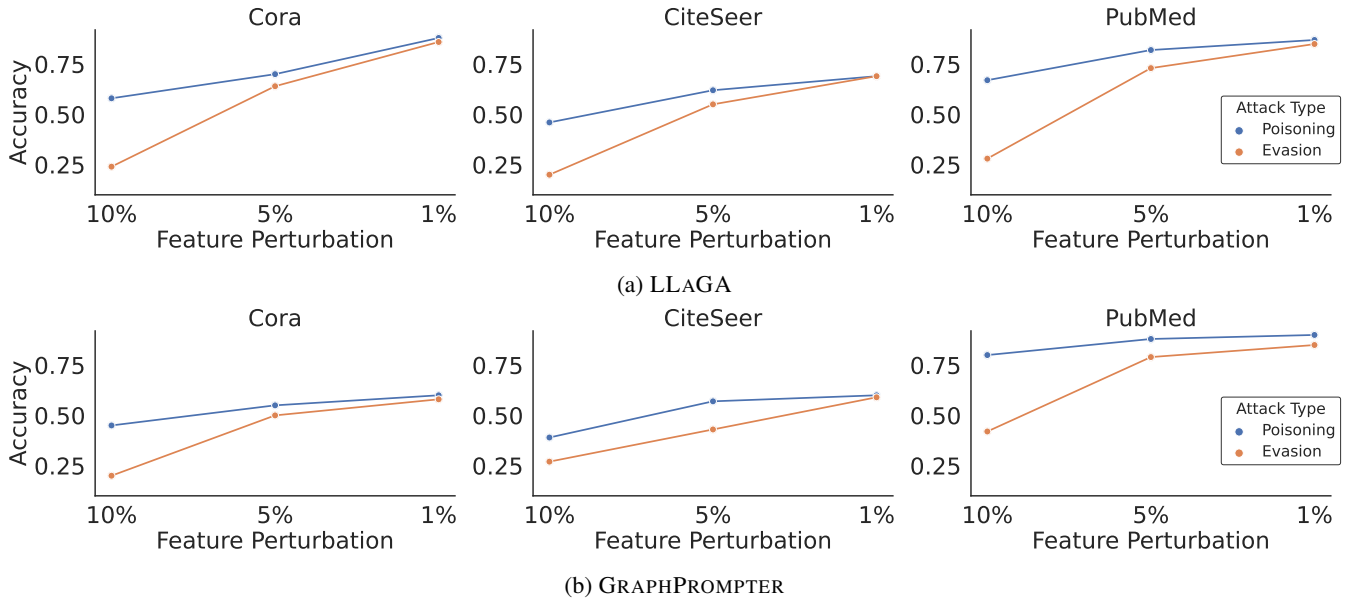


Figure 6: Accuracy of the graph-aware LLMs as the perturbed-feature fraction is varied. Lower accuracy indicates a more effective attack.

Table 8: Effect of the backbone LLM on clean accuracy and on accuracy under each attack, evaluated for LLAGA. The results indicate that the choice of backbone does not materially affect attack susceptibility.

Data	Base Model	Clean	Poisoning			Evasion		
			NETTACK	METAATTACK	MSI	NETTACK	METAATTACK	MSI
Cora	Vicuna-7B	0.89 ± 0.07	0.87 ± 0.04	0.79 ± 0.03	0.82 ± 0.04	0.55 ± 0.09	0.44 ± 0.06	0.30 ± 0.05
	Llama2-7B	0.90 ± 0.03	0.86 ± 0.02	0.77 ± 0.05	0.82 ± 0.02	0.56 ± 0.07	0.43 ± 0.08	0.31 ± 0.01
Citeseer	Vicuna-7B	0.71 ± 0.04	0.64 ± 0.05	0.63 ± 0.04	0.62 ± 0.02	0.59 ± 0.04	0.50 ± 0.03	0.24 ± 0.05
	Llama2-7B	0.71 ± 0.05	0.65 ± 0.03	0.60 ± 0.03	0.61 ± 0.04	0.58 ± 0.03	0.48 ± 0.05	0.26 ± 0.03
PubMed	Vicuna-7B	0.90 ± 0.04	0.89 ± 0.03	0.89 ± 0.08	0.81 ± 0.03	0.84 ± 0.05	0.73 ± 0.04	0.76 ± 0.03
	Llama2-7B	0.91 ± 0.06	0.88 ± 0.04	0.90 ± 0.02	0.80 ± 0.04	0.85 ± 0.03	0.75 ± 0.02	0.77 ± 0.02

an edge density of 0.81%, to assess the effectiveness of node sequence template attacks on large real-world graphs. As reported in Table 9, ArXiv exhibits a pronounced gap between the two modalities: poisoning consistently exceeds evasion across all three injection strategies, reversing the ordering observed on Cora, Citeseer, and PubMed. This suggests structural properties specific to ArXiv that render it more susceptible to poisoning yet more robust to evasion. Notably, despite its very low edge density, inserting even a single supernode is sufficient to materially degrade accuracy.

A.7 Feature Attack Examples and Defense Template

Feature Attack Examples The prompt examples in Appendix A.7 illustrate our attacks applied to real node features from the Cora dataset. The “input” denotes the perturbed text supplied to the LLM. The essential property is that, although a human reader perceives the original and perturbed text as visually identical or near-identical, the LLM ingests a distinct sequence of Unicode code points or a distinct character

ordering, which alters its behavior. Table 10 demonstrates this for the homoglyph attack and Table 11 for the reordering attack.

Feature Corrector Defense Template We designed the feature corrector module to counter the imperceptible attacks—specifically homoglyph substitutions and character reorderings—as a prompt-based defense driven by an LLM. The prompt instructs the LLM to act as a linguistic validator tasked with detecting and correcting subtle, human-imperceptible perturbations.

The prompt template is shown below. It states the task explicitly and prescribes a fixed output format, both of which measurably improve the LLM’s correction accuracy.

A.8 Detailed Discussion

Our findings yield several insights into the vulnerabilities and design trade-offs of graph-aware LLMs. As the integration of graph-structured data into LLMs continues to grow, characterizing not only their accuracy gains but also their

Prompt for Feature Corrector

You are a linguistic validator. Your task is to detect and correct any subtle, imperceptible text perturbations such as homoglyph substitutions (e.g., replacing "o" with "0") and character reorderings. These changes are often used to attack machine learning models. Given a text string, carefully analyze it. If you identify any perturbations, provide the corrected, original-looking version of the text. If no perturbations are found, return the original text unchanged.

Example 1 (Homoglyph Substitution):

Input: Title: The mgprir heuristic fr discvring prtin squnc pttrens Abstract: Svrl cmputr lgrithms fr discvring pttrens in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rlted squncs.

Output: Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Example 2 (Character Reordering):

Input: Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dscribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiottn of the apclpiation of three machine larening algorithms-lR, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.

Output: Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-lR, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

Example 3 (Mixed Homoglyph & Reordering):

Input: Title: Submitted to NIPS96, Sctin: Applications. Prfrnc: Oral prsentatin Reinforcement Lrning for Dynmic Channl Allocation in Abstract: In ellulr tlphne systms, n imrtnt prblm is to dynmiclly alloct th cmmunictin rsurc (chnnls) so as to mximize srvc in stchstic cllr nvirnmt. This prblem is naturally frmultd s dynmic prgrmming prblm and we us a reinforcement lerning (RL) mthd to find dynmic chnnl llction plicies that ar bttr than prvius heuristic slutins.

Output: Title: Submitted to NIPS96, Section: Applications. Preference: Oral presentation Reinforcement Learning for Dynamic Channel Allocation in Abstract: In cellular telephone systems, an important problem is to dynamically allocate the communication resource (channels) so as to maximize service in a stochastic caller environment. This problem is naturally formulated as a dynamic programming problem and we use a reinforcement learning (RL) method to find dynamic channel allocation policies that are better than previous heuristic solutions.

Your Task: Text to correct: {perturbed_text_here}

Corrected Text:

Figure 7: Prompt template for the LLM-based feature corrector module, specifying the LLM's role, task, and required output format.

Table 9: Node-classification accuracy under the node sequence template injection attack across datasets and injection variants, including ArXiv. *NI*, *SI*, and *MSI* denote the non-adjacent, single-supernode, and multiple-supernode injection variants, respectively; *Clean* denotes accuracy in the absence of any attack.

Data	Attack	NI	SI	MSI
Cora	Clean	————	0.89 ± 0.07	————
	Poisoning	0.86 ± 0.03	0.84 ± 0.06	0.82 ± 0.04
	Evasion	0.42 ± 0.08	0.36 ± 0.05	0.30 ± 0.05
Citeseer	Clean	————	0.71 ± 0.04	————
	Poisoning	0.69 ± 0.05	0.66 ± 0.03	0.62 ± 0.02
	Evasion	0.36 ± 0.06	0.30 ± 0.04	0.24 ± 0.05
PubMed	Clean	————	0.90 ± 0.04	————
	Poisoning	0.88 ± 0.04	0.85 ± 0.05	0.81 ± 0.03
	Evasion	0.85 ± 0.07	0.82 ± 0.04	0.76 ± 0.03
ArXiv	Clean	————	0.73 ± 0.02	————
	Poisoning	0.50 ± 0.03	0.43 ± 0.04	0.42 ± 0.03
	Evasion	0.71 ± 0.06	0.69 ± 0.02	0.60 ± 0.03

attack surface becomes essential. Our systematic evaluation of adversarial attacks on two representative graph-aware LLMs—LLAGA and GRAPHPROMPTER—exposes substantial weaknesses, most notably under the poisoning and evasion attacks conventionally used against GNNs. We discuss the principal observations below.

Impact of Graph Encoding on Adversarial Vulnerability Our central observation is that the mechanism used to integrate graph structure into an LLM can inadvertently increase its susceptibility to adversarial attack. Specifically, LLAGA’s node sequence template induces an exploitable vulnerability that yields substantial accuracy degradation. This attack surface—unique to LLAGA—exposes a fundamental design trade-off: node-level templates provide an effective encoding of graph structure and improve task accuracy, but simultaneously introduce attack vectors absent in conventional GNNs. In contrast, GRAPHPROMPTER, whose GNN-based projector encodes the graph, is comparatively more resilient, which points to the advantage of hybrid architectures that combine the complementary strengths of LLMs and GNNs.

Sensitivity of Graph-aware LLMs to Feature Perturbations Beyond the encoding-specific vulnerabilities, our results establish the effectiveness of feature perturbation attacks against both graph-aware LLMs. Despite their differing graph-encoding mechanisms, LLAGA and GRAPHPROMPTER are both markedly sensitive to imperceptible perturbations of the node features. This reflects a broader property of the LLM paradigm: although these models process textual data effectively, their robustness to adversarial perturbations—particularly those targeting node features—remains an open problem. The capacity of an adversary to degrade accuracy by subtly and undetectably altering node attributes undermines the integrity guarantees of graph-aware LLMs, especially in security-sensitive deployments. Text-sanitization techniques such as those of (Boucher et al. 2022) offer partial

mitigation, but robustly countering imperceptible manipulations such as homograph substitution remains difficult. Our proposed LLM-based corrector addresses this by prompting the model to detect and repair suspicious or malformed inputs prior to downstream processing.

Data-dependent Variability of Poisoning and Evasion Attacks on Graph-aware LLMs Our results show that the efficacy of poisoning and evasion attacks on graph-aware LLMs is dataset-dependent rather than uniform. The attacks are highly effective on Cora, Citeseer, and PubMed, whereas ArXiv exhibits a more intricate interaction between attack modality and graph structure. This variation underscores the role of dataset characteristics in robustness evaluation: as graph-aware LLMs are deployed across diverse domains, their vulnerabilities must be assessed with respect not only to architectural design but also to the specific data distribution to which they are applied.

Implications for Graph-aware LLM Development The vulnerabilities characterized in this study bear directly on the future development and deployment of graph-aware LLMs. Given their growing adoption for objectives traditionally addressed by GNNs, the trade-off between predictive accuracy and adversarial exposure must be treated as a first-class design consideration. As these architectures evolve, there is a pressing need for more robust graph-encoding mechanisms and for defenses against adversarial manipulation, particularly feature perturbations. Moreover, characterizing how adversarial attacks transfer from GNNs to graph-aware LLMs opens new directions for defending these models against sophisticated adversaries.

Difficulty in Applying Existing GNN defenses Against Adversarial Attacks on Graph-aware LLMs **Defenses on adversarial attacks based on feature perturbation.** Adversarial attacks on GNNs have concentrated on structural perturbations, since feature-level perturbations typically fail to flip the predicted class of the target node (Dai et al. 2018; Wu et al. 2019; Zügner and Günnemann 2019) and are therefore ineffective as attacks. As a consequence, there exist *no known defenses against feature-based attacks on GNNs*. In the graph-aware LLM paradigm, however, feature perturbations are effective, as demonstrated in the main paper. This poses a novel challenge: attacks that were inconsequential against GNNs can now manipulate model behavior, which necessitates defenses covering both structural and feature-level perturbations. Text-sanitization techniques—for instance, applying optical character recognition (OCR) as a preprocessing step before encoding text for the LLM—offer partial mitigation, but robustly countering imperceptible manipulations such as homograph substitution remains difficult, since OCR is imperfect and misreads homographs at a higher rate than unperturbed text (Boucher et al. 2022). These limitations motivated our LLM-based self-correcting mechanism for feature perturbations.

Defenses on adversarial attacks based on structural perturbation. Because structure-based adversarial attacks trans-

Table 10: Illustration of the imperceptible homoglyph attack. The LLM ingests the perturbed text as a distinct code-point sequence, whereas the rendered output is visually near-identical for a human reader, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: The megaprior heuristic for discovering protein sequence patterns Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.	Title: The mgpprir huristic fr discvring prtin squnc pptrns Abstract: Svrl cmputr lgrithms fr discvring pptrns in grups f prtin squncs r in us tht r bsd on fitting th prmtrs f sttisticl mdl to grup of rltd squncs.	Title: The megaprior heuristic for discovering protein sequence patterns. Abstract: Several computer algorithms for discovering patterns in groups of protein sequences are in use that are based on fitting the parameters of a statistical model to a group of related sequences.

Table 11: Illustration of the imperceptible reordering attack. The LLM ingests the perturbed text as a distinct character ordering, whereas the rendered output is visually near-identical for a human reader, as shown in the image.

Original Feature	What LLM Sees	What a Human Sees
Title: Applications of machine learning: a medical follow up study. Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.	Title: Appliatcions of machine lreanign: a medical foall-up study Abstract: This ppaer dsceribes pmerlinary work that aims to aplpy smoe learning sttraegies to a meidcal flolow-up study. An inestivagiotn of the aplcpiation of three machine larening algorithms-1R, FOIL and InductH to identify risk frcatos that govern the clopsuspension cure rtae has been mdae.	Title: Applications of machine learning: a medical follow up study Abstract: This paper describes preliminary work that aims to apply some learning strategies to a medical follow-up study. An investigation of the application of three machine learning algorithms-1R, FOIL and InductH to identify risk factors that govern the colposuspension cure rate has been made.

fer from GNNs to graph-aware LLMs—particularly under evasion—one might expect GNN-derived defenses to remain effective in this paradigm. This expectation does not hold: unlike the standard GNN setting, *graph-aware LLMs are additionally susceptible to feature perturbations, which renders these defenses ineffective*.

One representative GNN defense makes the adjacency matrix trainable so that edge weights adapt during training to counter adversarial perturbations, assigning lower weights to edges between dissimilar nodes and higher weights to edges between nodes of greater feature similarity. Several **preprocessing-based** defenses build on this principle, e.g., deleting edges between nodes of low feature similarity (Wu et al. 2019) or applying randomized edge dropping (Dai et al. 2018). All such defenses, however, depend on clean, unperturbed node features to compute reliable similarity scores;

when the features are themselves perturbed, as under our attacks, the defenses become ineffective and may even worsen matters by reconstructing incorrect edges. We illustrate this failure mode on a randomly selected node of the Cora dataset (Figure 8): the similarity graph computed from clean features preserves some original edges while adding some spurious ones, whereas the graph reconstructed from perturbed features preserves none of the original edges, yielding an entirely incorrect reconstruction.

A second line of work reduces reliance on the graph structure. (Shanthamallu, Thiagarajan, and Spanias 2021) trains a surrogate predictor that depends solely on node features and uses an uncertainty-matching strategy to distill graph information from the GNN, while Wu et al. (2021) employs a co-training framework in which submodels are trained independently on the feature and structural views, with knowledge

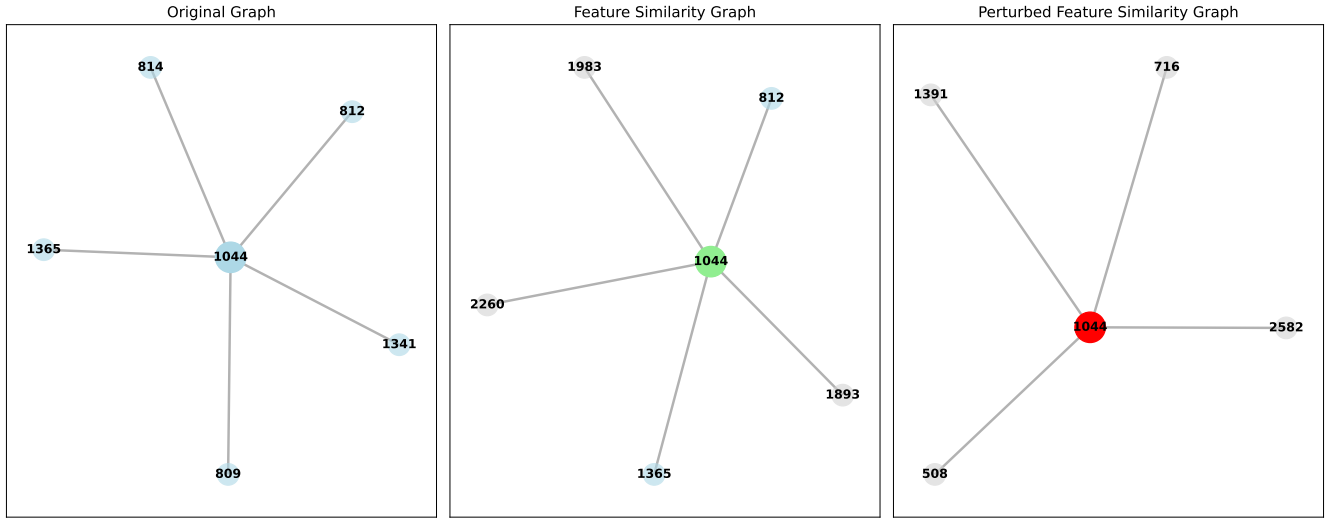


Figure 8: Reconstructed edge sets for a randomly selected node (node 1044). **Original (left):** the ground-truth edges from the original graph. **Feature Similarity (middle):** edges reconstructed from clean node features, with blue nodes marking connections preserved from the original graph. **Perturbed Feature Similarity (right):** edges reconstructed from perturbed node features.

distillation via the inclusion of confidently labeled unlabeled data. Both approaches again presuppose access to unperturbed node features—an assumption violated in our setting, where structure and features are perturbed jointly. Consequently, GNN-derived structural defenses do not transfer directly to the graph-aware LLM paradigm, which demands defenses tailored to it.